MAS1004 Data & AI, 2026 Fall

# Assignment 1: Classifying Korean album covers by release era

Name: Hyewon Shin Student ID: 20261220

## Problem 1. Categories and images

### Why these categories

I chose Korean album covers from four eras: the 1980s–1990s, 2000s, 2010s, and 2020s. I enjoy music and am interested in cultural and social change, so I wanted to see whether cover designs distinguish release eras. I combined the 1980s and 1990s because I expected limited data for the 1980s and possibly similar visual styles.

### What I expected the model to use

Before training, I expected colour, resolution and scan traces, typography, layout, and photographic or graphic styles to provide clues. I was concerned that the model might rely on superficial correlations such as low image quality rather than meaningful era-related design.

### Images per category

| Era | Initial dataset | Final CLEAN dataset | New images |
|---|---|---|---|
| 1980s–1990s | 146 | 93 | 8 |
| 2000s | 144 | 79 | 6 |
| 2010s | 167 | 87 | 8 |
| 2020s | 175 | 114 | 5 |
| Total | 632 | 373 | 27 |

Table 1: Readable images before splitting, and images from a new source.

## Problem 2. Preparing an image

`prepare_image()` converts to RGB, resizes the shorter side to 256 using bilinear interpolation, takes the central $224 \times 224$ crop, divides pixels by 255, and normalizes with RGB means $(0.485, 0.456, 0.406)$ and standard deviations $(0.229, 0.224, 0.225)$. It returns a `float32` array of shape $(3, 224, 224)$.

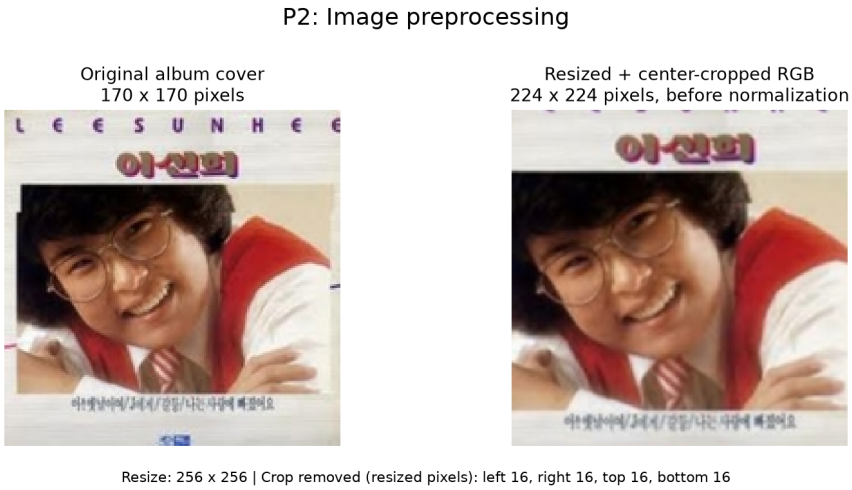


Figure 1: Original cover (left) and resized, centre-cropped RGB view before normalization (right).

The example is `data/clean/1980s_1990s_korean_album_covers/1985.jpg`: 170×170 becomes 256×256, then 224×224, removing 16 pixels from each edge of the resized image. Parts of the upper lettering, lower logo and margins disappear, and the face occupies more of the frame. The saved asset audit confirms that inverse-normalizing the prepared tensor reproduces the RGB crop.

I noticed that cropping can make the original layout harder to recognize. I accepted this trade-off because colour, lettering, illustration and photographic style remain visible. However, I did not identify a specific example in which an important subject part was lost; this evidence only documents edge and lettering loss.

## Problem 3. First training runs

| Tag | Start | Trained | Epochs | lr | Trainable parameters | Train acc. | Test acc. |
|---|---|---|---|---|---|---|---|
| `run` | ImageNet | All layers | 10 | 0.0001 | 11,178,564 | 94.1% | 46.8% |
| `scratch` | Random | All layers | 10 | 0.0001 | 11,178,564 | 93.3% | 50.8% |
| `frozen` | ImageNet | Last layer | 10 | 0.001 | 2,052 | 71.9% | 43.7% |
| `lr3e4` | ImageNet | All layers | 10 | 0.0003 | 11,178,564 | 94.1% | 54.0% |
| `clean` | ImageNet | All layers | 10 | 0.0001 | 11,178,564 | 100.0% | 56.0% |

Table 2: Final evaluation values printed by each run. The first four runs preceded cleaning; CLEAN is discussed in Problem 4.

### Loss and the train–test gap

| Tag | First train loss | Last train loss | Test-loss trend |
|---|---|---|---|
| `run` | 1.2815 | 0.1366 | Initially fell to 1.1517, then rose to 1.3818. |
| `scratch` | 1.3574 | 0.2164 | Fluctuated; ended at 1.7261, above its initial value. |
| `frozen` | 1.4338 | 0.8968 | Decreased slowly and flattened late in training. |
| `lr3e4` | 1.3223 | 0.1233 | Fluctuated rather than following the train-loss decrease. |
| `clean` | 1.4018 | 0.0113 | Flattened late in training. |

Table 3: Loss trends from the saved training outputs and curves.

Training loss decreased in every run, with smaller improvements late in training. The final train–test gaps were 47.3, 42.5, 28.2, 40.1 and 44.0 percentage points for `run`, `scratch`, `frozen`, `lr3e4` and `clean`, respectively. I interpret these large gaps and the test-loss plateaus or increases as evidence of overfitting.

### Initialization and trained layers

Random initialization scored 4.0 percentage points above ImageNet initialization on the same data and epoch count. I think ImageNet features may not align closely with cover-era distinctions, while the small dataset and optimization variability may also matter; one run does not establish the cause.

Training only the last layer scored 3.1 percentage points below full fine-tuning while updating 2,052 rather than 11,178,564 parameters. It retained some useful classification ability, but its different learning rate prevents a strictly layer-only comparison. My chosen learning rate, 0.0003, scored 7.2 percentage points above `run`; I did not repeat these runs.

### Confusion and my expectation

In `run`, the largest one-way confusion was 2020s → 2010s (12 images); 2010s → 2000s and 2020s accounted for 5 and 4 images. In CLEAN, the largest was 2010s → 2020s (10 images), with 3 going from the 2010s to the 2000s.

I expected the model to confuse the 2010s with both neighboring eras because I saw it as a period of substantial change. The errors partly match that expectation, although the largest initial error went in the opposite direction.

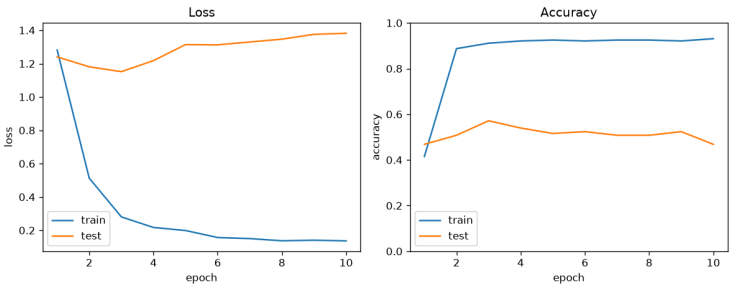



Figure 2: `run`: loss and accuracy.

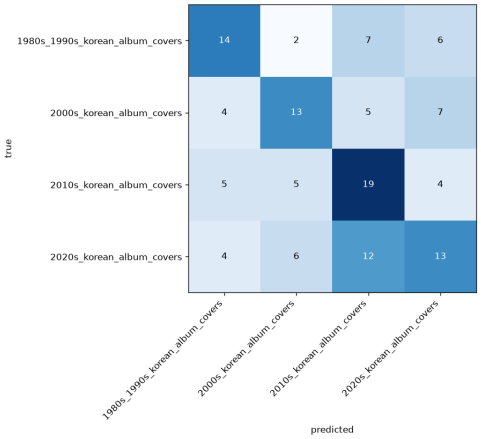



Figure 3: `run`: test confusion matrix.

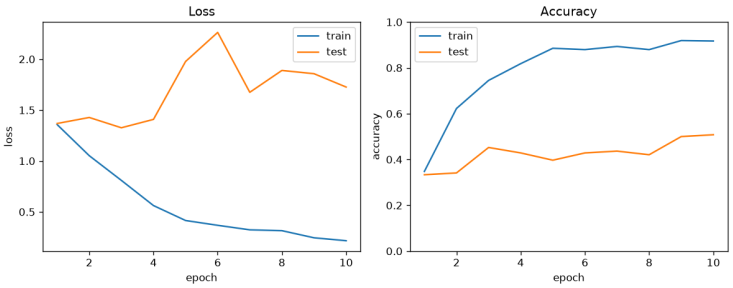



Figure 4: `scratch`: loss and accuracy.

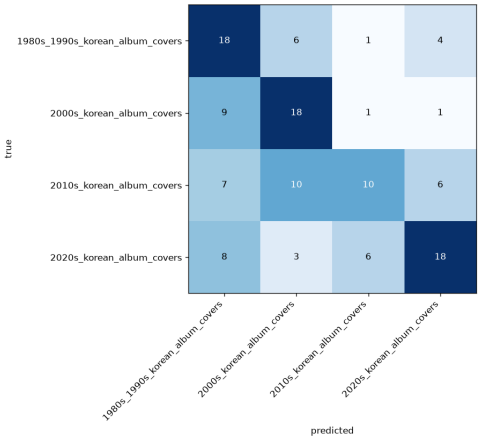



Figure 5: `scratch`: test confusion matrix.

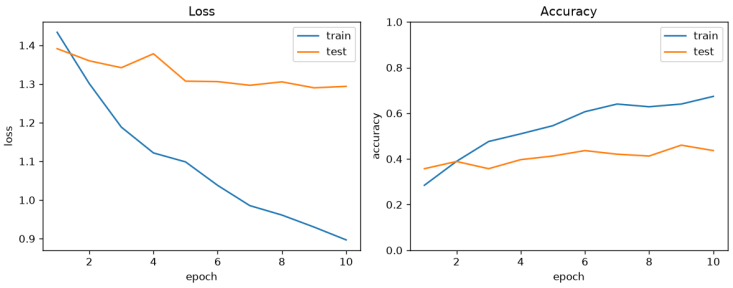



Figure 6: `frozen`: loss and accuracy.

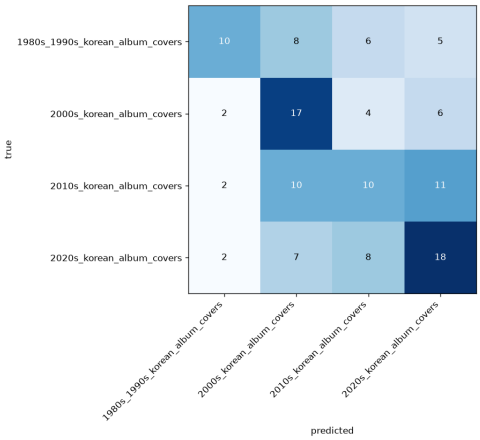



Figure 7: `frozen`: test confusion matrix.

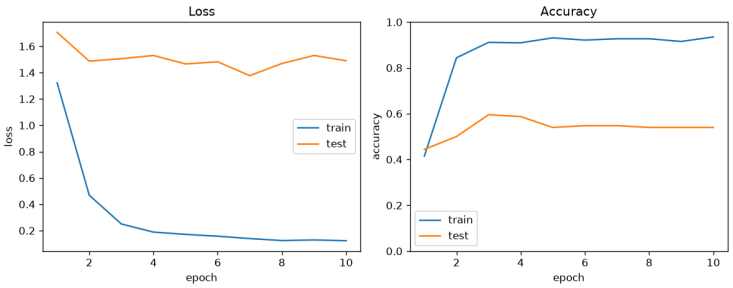



Figure 8: `lr3e4`: loss and accuracy.

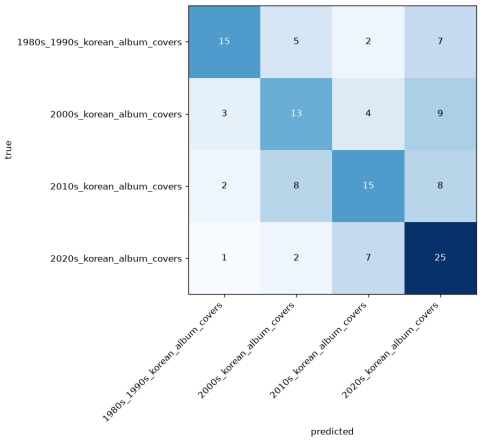



Figure 9: `lr3e4`: test confusion matrix.

## Problem 4. Cleaning

### My rule

I decided to exclude large collages, non-cover images such as promotional photographs or thumbnails, covers that were difficult to identify because of the viewing angle, foreign albums, and duplicate covers. Era mismatch was added later as a removal reason.

After cleaning, I found performance lower than I expected and manually supplemented the dataset with higher-quality covers selected by artist and era from music websites, with my professor's approval. The exact addition count is not recorded, so CLEAN measures cleaning plus supplementation rather than cleaning alone.

### Three kinds of junk

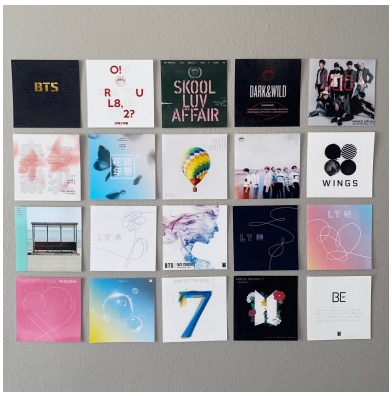



Figure 10: Large collage: `data/removed/2000s_korean_album_covers/q22_0005.jpg`.

Several covers appear together rather than as one identifiable cover.

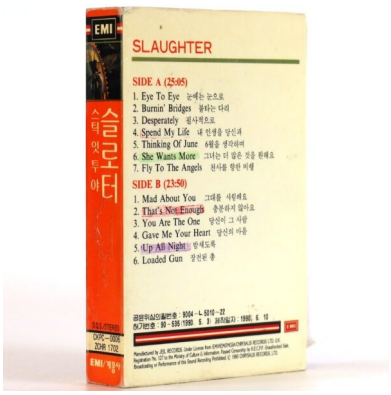



Figure 11: Non-cover image: `data/removed/1980s_1990s_korean_album_covers/q03_0022.jpg`.

The back and side of Slaughter cassette packaging and its track list dominate the image; the logged reason was "not an album cover."




Figure 12: Hard-to-identify cover: `data/removed/1980s_1990s_korean_album_covers/q19_0022.jpg`.

An angled photograph includes an LP, packaging and printed material, obscuring a single front cover.

**What I removed**

| Logged reason | 1980s–1990s | 2000s | 2010s | 2020s | Total |
|---|---|---|---|---|---|
| Large collage | 0 | 3 | 3 | 0 | 6 |
| Not a cover | 3 | 7 | 3 | 5 | 18 |
| Hard to identify | 2 | 21 | 4 | 0 | 27 |
| Foreign album | 2 | 12 | 10 | 7 | 31 |
| Duplicate cover | 6 | 7 | 8 | 5 | 26 |
| Era mismatch | 0 | 9 | 22 | 7 | 38 |
| Within-class duplicate | 13 | 12 | 12 | 13 | 50 |
| Total | 26 | 71 | 62 | 37 | 196 |

Table 4: Cumulative removal-log entries, from `outputs/removal_by_class.csv`.

The log covers an initial 85 removals, 82 later entries that day, and 29 the next day. The saved `clean.py count` below describes only the initial stage, retaining 133/122/136/156 files. Its `other` column means files present in raw but absent from both clean and the removal log. Unlogged manual removals and supplementation prevent a complete count reconciliation with the final readable dataset.

```
class                    downloaded  removed   kept  other
1980s_1990s_korean_album_covers       1350       13    133   1204
2000s_korean_album_covers        690        22    122    546
2010s_korean_album_covers        690        31    136    523
2020s_korean_album_covers        510        19    156    335

reasons
    22  중복된 앨범 커버이므로 정제 기준에 따라 제외합니다.
    21  외국 앨범 커버이므로 한국 앨범 커버 정제 기준에 따라 제외합니다.
    18  분류된 시대와 맞지 않는 앨범 커버이므로 정제 기준에 따라 제외합니다.
    12  앨범 커버가 아닌 이미지이므로 정제 기준에 따라 제외합니다.
     7  앨범 커버를 식별하기 어려운 이미지이므로 정제 기준에 따라 제외합니다.
     5  대형 콜라주 이미지이므로 정제 기준에 따라 제외합니다.

"other" is images that are in data/raw but neither in data/clean nor removed with `clean.py
↪  remove`, for example ones you deleted by hand. Their reasons are not recorded.
```

The saved suspects CSV contains 632 ranked images and 183 near-copy *pairs*, not 183 groups or removed files. Matching its top 20 entries per class to the cumulative removal log gives:

| Era | 1980s–1990s | 2000s | 2010s | 2020s |
|---|---|---|---|---|
| Top-20 suspects removed | 7 | 12 | 8 | 9 |

An earlier summary reported 7 rather than 12 for the 2000s; its counting stage is undocumented. I use the cumulative audit here. I do not have specific recorded examples of junk I first missed but found through the suspects list, or junk absent from that list.

An earlier audit reported 63 byte-identical groups (17 cross-class, 46 within-class), with 27 cross-class and 50 within-class files selected for removal. These historical figures are not the same count as the 26 "duplicate cover" log entries. The saved checker found no byte-identical duplicates, but the original group lists and decisions for all 183 near-copy pairs are unavailable; I cannot claim that every near copy was resolved.

**Before and after cleaning**

| Model | Downloaded test accuracy | Accuracy on the same new images |
|---|---|---|
| `run` | 46.8% | 29.6% (8/27) |
| `clean` | 56.0% | 51.9% (14/27) |

Table 5: Before and after cleaning and supplementation.

Neither final accuracy decreased. Cleaning changed the downloaded test split, whereas the same 27 new images provide a direct model comparison. I cannot separate the effects of cleaning and supplementation.

The following comparison predates export, so its missing-web-files warning is historical.

```
MAS1004 Assignment 1, checking your work

Training images  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean)
------------------------------------------------------------------------
  1980s_1990s_korean_album_covers    93 images
  2000s_korean_album_covers       79 images
  2010s_korean_album_covers       87 images
  2020s_korean_album_covers      114 images

Your images from a new source  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\new_images)
------------------------------------------------------------------------------------------
  1980s_1990s_korean_album_covers     8 images
  2000s_korean_album_covers        6 images
  2010s_korean_album_covers        8 images
  2020s_korean_album_covers        5 images

The same image appearing twice
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

The model you exported for the web page
---------------------------------------
  PROBLEM  docs/ is missing model.onnx, model.json, selftest.json. Run src/run.py and then
  ↪  src/export_web.py first.

Your saved runs on your new images
↪  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\new_images)
------------------------------------------------------------------------------------------------」
↪  --
  run               test accuracy   new images
  run                       46.8%        29.6%   (8/27)
  clean                     56.0%        51.9%   (14/27)

Summary
-------
  1 thing(s) to fix:
    - docs/ is missing model.onnx, model.json, selftest.json. Run src/run.py and then
    ↪  src/export_web.py first.
```

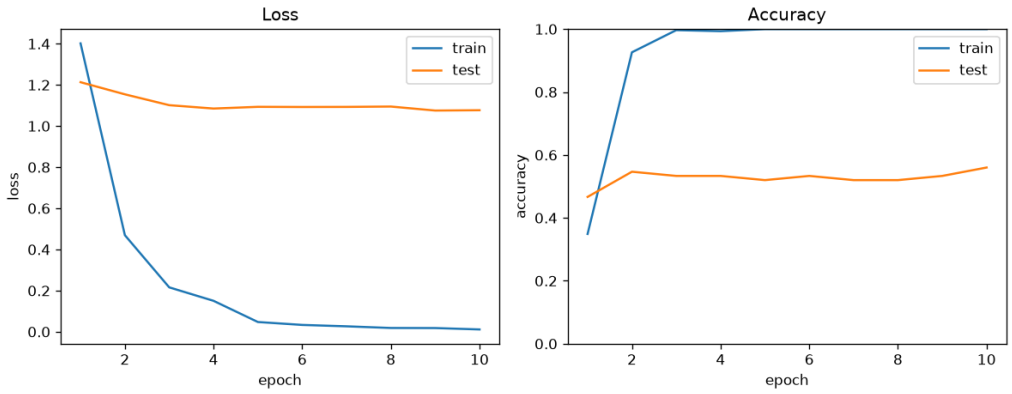



Figure 13: CLEAN: loss and accuracy.

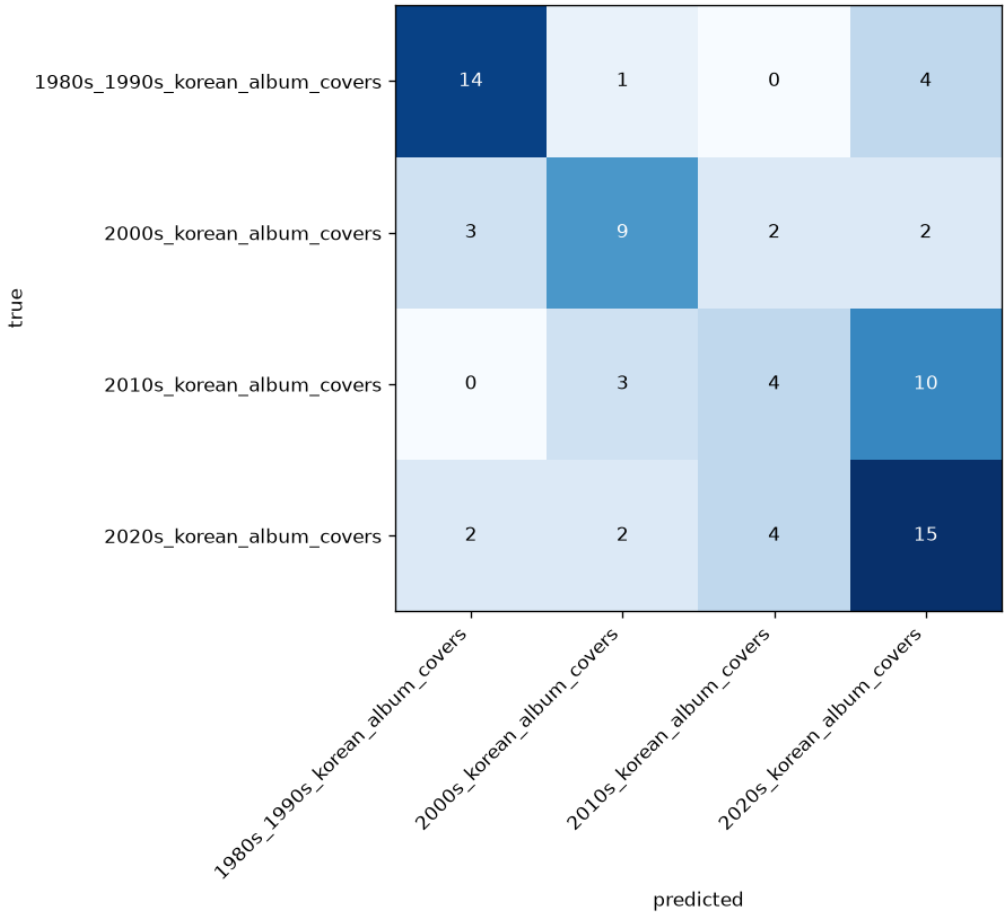



Figure 14: CLEAN: test confusion matrix.



Figure 15: CLEAN's most confident downloaded-test mistakes.

## Output of the CLEAN run

```
reading C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean
373 images, 4 classes, read in 11 s
  1980s_1990s_korean_album_covers 93
  2000s_korean_album_covers 79
  2010s_korean_album_covers 87
  2020s_korean_album_covers 114
training on 298, testing on 75
model has 11,178,564 parameters, 11,178,564 of them will be trained
Epoch 1/10: train loss 1.4018, train accuracy 0.3490, test loss 1.2131, test accuracy 0.4667
Epoch 2/10: train loss 0.4693, train accuracy 0.9262, test loss 1.1542, test accuracy 0.5467
Epoch 3/10: train loss 0.2158, train accuracy 0.9966, test loss 1.1014, test accuracy 0.5333
Epoch 4/10: train loss 0.1503, train accuracy 0.9933, test loss 1.0851, test accuracy 0.5333
Epoch 5/10: train loss 0.0474, train accuracy 1.0000, test loss 1.0934, test accuracy 0.5200
Epoch 6/10: train loss 0.0332, train accuracy 1.0000, test loss 1.0927, test accuracy 0.5333
Epoch 7/10: train loss 0.0264, train accuracy 1.0000, test loss 1.0931, test accuracy 0.5200
Epoch 8/10: train loss 0.0184, train accuracy 1.0000, test loss 1.0948, test accuracy 0.5200
Epoch 9/10: train loss 0.0180, train accuracy 1.0000, test loss 1.0753, test accuracy 0.5333
Epoch 10/10: train loss 0.0113, train accuracy 1.0000, test loss 1.0769, test accuracy 0.5600

trained in 5.2 minutes
train accuracy 100.0%
test  accuracy 56.0%
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\clean_curves.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\clean_confusion.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\clean_worst.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\clean_model.pt

One line for your experiment table:
| clean | ImageNet | all layers | 10 | 0.0001 | 11,178,564 | 100.0% | 56.0% |

To put this model on your web page: python src/export_web.py --tag clean
```

### Problem 5. Images from a new source

I manually selected era-appropriate artists and albums from music websites and checked against my downloads. The class counts in Problem 1 meet the minimum of five each. However, I did not preserve site URLs or evidence that the downloader could not have found these sources, so source independence remains unverified.

The saved overlap check compared the 27 new images with 1,682 downloaded images and found no similarity above 0.95; the closest was 0.85. This establishes no detected near copies within that check, not source independence.

```
describing 1709 images with ResNet18 ...

none of your 27 new images is a near copy of any of the 1682 images you downloaded

the closest any new image comes to a downloaded one: 0.85 (near copies are above 0.95)
```

The CLEAN accuracies appear side by side in Problem 4: the new-source score is lower than the downloaded-test score. I think source differences and the small evaluation sample may contribute. The new-source confusion matrix is:

| Actual \ Predicted | 1980s–1990s | 2000s | 2010s | 2020s |
|---|---|---|---|---|
| 1980s–1990s | 5 | 1 | 1 | 1 |
| 2000s | 1 | 4 | 1 | 0 |
| 2010s | 0 | 3 | 2 | 3 |
| 2020s | 2 | 0 | 0 | 3 |

Table 6: CLEAN on new images; rows are actual eras and columns are predictions.

#### Five confident mistakes

These are my interpretations, not isolated causes. Paths below are relative to `data/new_images/`; WebP examples were converted to PNG only for report display.




Figure 16: `2010s_korean_album_covers/2013.webp`: actual 2010s, predicted 2000s (96%).

I think its monochrome-like aesthetic and freely handwritten-looking English typography resemble 2000s covers. The file itself contains colour.

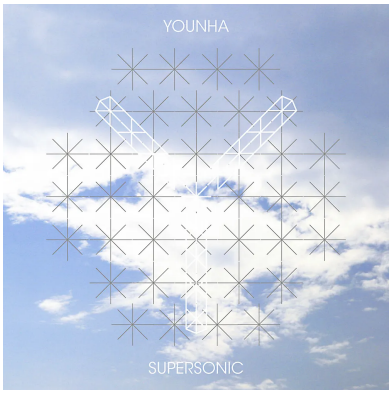



Figure 17: `2010s_korean_album_covers/2012'.webp`: actual 2010s, predicted 2020s (89%).

I think the simple sky background and repeated linear design may look like a polished 2020s layout.



Figure 18: `1980s_1990s_korean_album_covers/1985.jpg`: actual 1980s–1990s, predicted 2000s (87%).

I could not find a convincing explanation for this mistake. This is not the image with the same filename in Problem 2.




Figure 19: `2010s_korean_album_covers/2012.webp`: actual 2010s, predicted 2020s (82%).

I identified this as PSY's Gangnam Style cover. Although the illustration reminds me of its era, I think the vivid colours may have pushed the prediction toward the 2020s.




Figure 20: `2010s_korean_album_covers/2014.webp`: actual 2010s, predicted 2000s (72%).

I think its yellowish tone and monochrome-like aesthetic may have contributed to the 2000s prediction; it is not literally grayscale.

#### How the new images differ

The downloaded examples include collages on walls, cassette packaging, and angled LP photographs with surrounding cardboard or inserts. CLEAN's test mistakes also include angled packaging and exposed CDs. In contrast, these new examples fill the frame with the cover design, without surrounding desks or packaging.

Physical photographs introduce viewing distance, margins and perspective; those external camera conditions are not identifiable in these digital-cover examples. Their lighting and colour are part of the cover artwork, not a known new photography setup. This comparison describes the inspected examples, not the full distribution. Some new files are WebP, contrary to README's JPEG/PNG instruction, although the saved checker read them successfully.

### Problem 6. The demo

Address: `https://downtownwonn.github.io/mas1004-assignment1-new/`

I confirmed that the demo worked; the saved Problem 7 screenshots show predictions and the model's crop. A green self-test badge and a phone test away from my computer are not documented. The saved checker reports Python–ONNX agreement (largest displayed gap 0.0000); its uncommitted-model warning is from that earlier check, not a verified current deployment diagnosis.

When I uploaded an *agujjim* food photograph, it chose the 2020s; for my close-up portrait photograph, it chose the 2000s, because the classifier has no "not an album cover" option. I did not preserve screenshots of these two inputs.

### Problem 7. What the model looks at

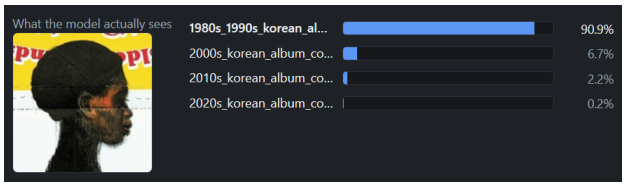
The evidence is the saved web screenshots in `results/P5_experiments/`. Every probability vector below uses the order [1980s–1990s, 2000s, 2010s, 2020s], in percent, preserving displayed rounding. The screenshots show the model's crop. Exact requests remain in their original Korean, including typos, as required.

#### Guess 1. The black circular area resembles an LP

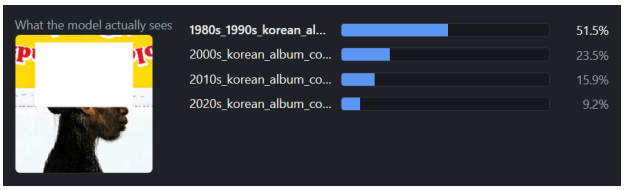
I thought the black circular area on a 2000s cover might resemble an LP and explain its older-era prediction. I chose to cover it and check whether the older-era probability fell.

My requests to the agent, word for word:

> “첫번째 사진 가리는거”
> “회색말고 흰색으로 가려주고, 검은 부분을 더 가려주라”
> “원형말고 네모낳게 가려주라”




`1_original.png`: [90.9, 6.7, 2.2, 0.2]




`1_modified.png`: [51.5, 23.5, 15.9, 9.2]

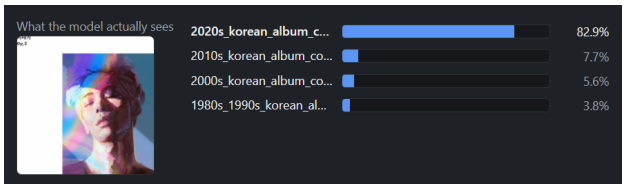
Figure 21: Covering the black area with a white rectangle.

The older era remained the top prediction, but its probability fell. This is consistent with my guess, without confirming it: the rectangle also covers part of the head and face and introduces a white region.

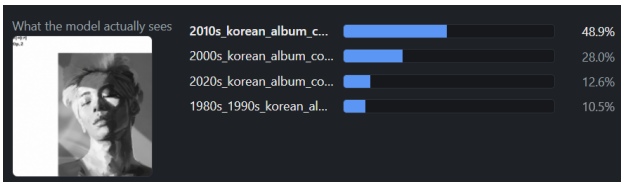
### Guess 2. Colour contributes to the 2020s prediction

My initial colour hypothesis and a confident 2020s mistake led me to remove colour while requesting the same proportions and layout. I then tested the same question on another cover.
My requests to the agent, word for word:

> “자 이번엔 두번째 사진, 원이미지 비율 레이아웃 그대로 유지하면서 무채색으로만 바꿔줘”



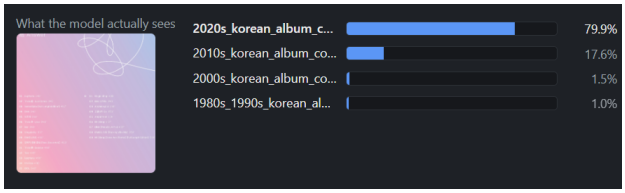

`2_original.png`: [3.8, 5.6, 7.7, 82.9]



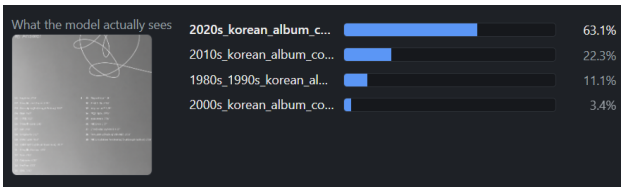

`2_modified.png`: [10.5, 28.0, 48.9, 12.6]

Figure 22: First grayscale change.

> “이번엔 세번째 사진, 원이미지 비율 레이아웃 그대로 유지하면서 무채색으로만 바꿔줘”




`3_original.png`: [1.0, 1.5, 17.6, 79.9]




`3_modified.png`: [11.1, 3.4, 22.3, 63.1]

Figure 23: Grayscale change on a second independent original.

The first prediction changed to the 2010s; the second remained in the 2020s with lower probability. I interpret this as colour-related sensitivity that varies by image, not a rule that grayscale implies an older era. The generative edits may also alter brightness or texture.

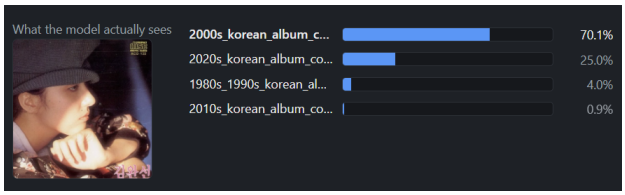
### Further tests on a misclassified portrait cover

The original 1980s–1990s portrait cover was classified as 2000s. I explored direction, expression and accessories as possible clues. The following records preserve my changes and observations, but not a specific advance prediction for each change. The front-view, smile and cap-removal comparisons are cumulative, not separate edits of the original.

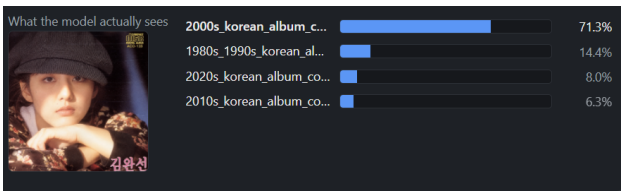
### Change 4. Front-facing portrait

I changed the direction to see whether the prediction responded.

"네번째 사진, 원이미지 비율 레이아웃 그대로 유지하면서 정면으로만 바꿔줘"




`4_original.png`: [4.0, 70.1, 0.9, 25.0]




`4_modified1_front_view.png`: [14.4, 71.3, 6.3, 8.0]

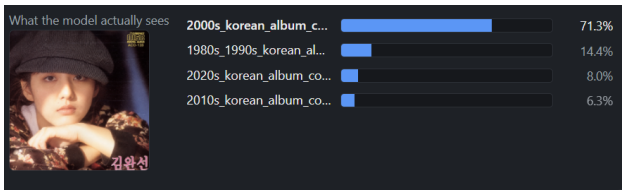
Figure 24: Front-facing edit.

The top prediction stayed in the 2000s. The hands and arms also changed, so this was not a direction-only test.

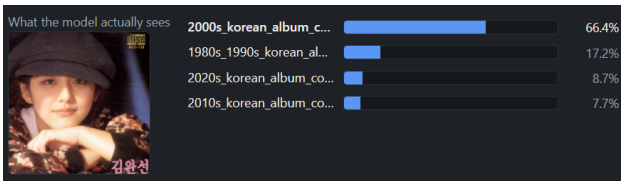
### Change 5. Adding a smile

I added a smile to the front-facing version to explore expression.

"이번엔 정면 흐뭇하게 웃는 얼굴로 바꿔줘 나머지는 원본 유지하면서"




Before smile: [14.4, 71.3, 6.3, 8.0]




`4_modified2_smile.png`: [17.2, 66.4, 7.7, 8.7]

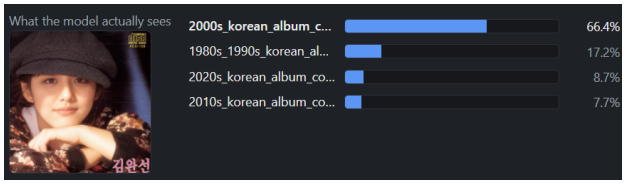
Figure 25: Smile added to the front-facing version.

The 2000s probability decreased, but the top class did not change.

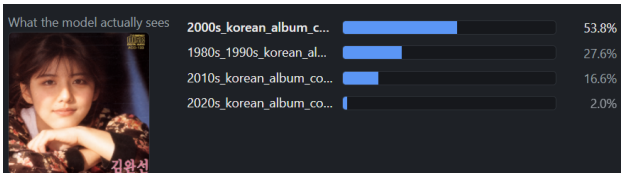
### Change 6. Removing the beret

I removed the beret from the smiling version to explore the accessory's influence.

"이번엔 정면에 흐뭇하게 웃는 얼굴에 추가로 베레모를 벗겨줘"




Before cap removal: [17.2, 66.4, 7.7, 8.7]

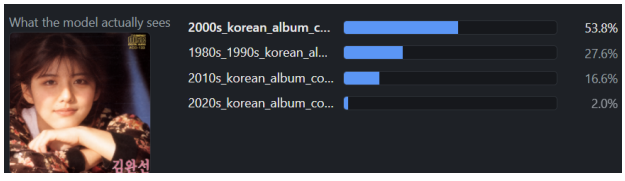



`4_modified3_nocap.png`: [27.6, 53.8, 16.6, 2.0]

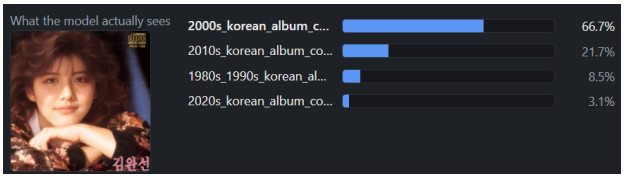
Figure 26: Beret removed from the smiling version.

The 2000s probability decreased and the older-era probability increased. Removing the cap also generated previously hidden hair, so I cannot isolate the cap's effect.

### Additional observation. Fuller wavy hair




Capless comparison: [27.6, 53.8, 16.6, 2.0]




`4_modified4_hair.png`: [8.5, 66.7, 21.7, 3.1]

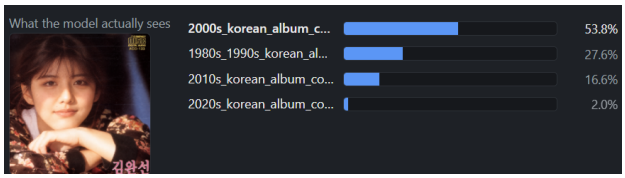
Figure 27: Preserved fuller, wavy-hair version.

The saved version has fuller wavy hair, but my exact request is unavailable. I include the evidence without treating it as a fully documented independent test.

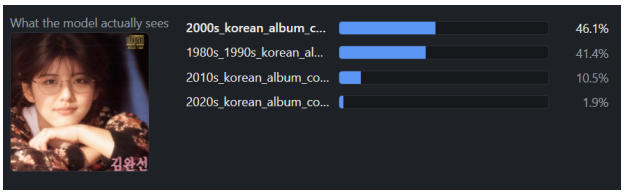
### Change 7. Metal-framed glasses

I requested glasses twice; only one confirmed result screenshot remains.

> “이번엔 금속테 안경을 씌워줘”




Visually corresponding capless version: [27.6, 53.8, 16.6, 2.0]




`4_modified5_glasses.png`: [41.4, 46.1, 10.5, 1.9]

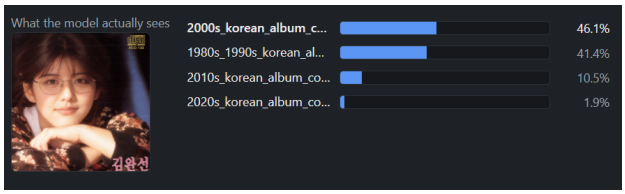
Figure 28: Glasses version and its visually corresponding comparison.

Relative to this comparison, the 2000s probability fell and the older-era probability rose. The hair resembles the capless version rather than the wavy-hair edit. I cannot confirm the actual input file or which request produced the result, so I do not treat these files as one linear sequence or two glasses tests.

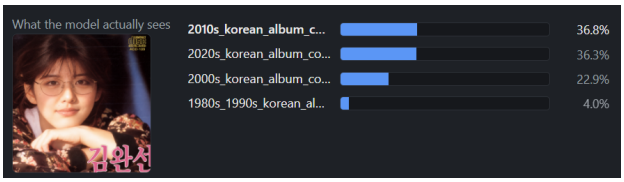
### Change 8. Enlarging the singer’s name

I expected a more prominent singer’s name might help the model recognize the older era. I requested:

> “이번엔 우측하단 김완선 글자를 두배크기로 키워줘”




Visually corresponding glasses version: [41.4, 46.1, 10.5, 1.9]




`4_modified6_bigletter.png`: [4.0, 22.9, 36.8, 36.3]

Figure 29: Enlarged-name version and its visually corresponding comparison.

Contrary to my expectation, the older-era probability fell and the 2010s became the top class, narrowly ahead of the 2020s. The actual edit input is not documented; this does not show that the model read the name or that text size alone caused the change. “Twice” describes my request, not a measured output size.

### What I now think the model uses

I tentatively think colour, layout, portrait direction, expression and accessories can influence predictions. My strongest observation is that the selected covers are sensitive to edits; I have not identified a feature that determines the release era. Generative changes and cumulative baselines limit causal interpretation.

Only grayscale was tested on two independent originals. I did not document several independent originals for each other change, all advance hypotheses and expected directions, the hair request, or the exact glasses/text input chain. These are remaining gaps against Problem 7, not additional completed tests.

## Appendix. Output of check.py

### Output of the runs

```
reading C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean
632 images, 4 classes, read in 15 s
  1980s_1990s_korean_album_covers 146
  2000s_korean_album_covers 144
  2010s_korean_album_covers 167
  2020s_korean_album_covers 175
training on 506, testing on 126
model has 11,178,564 parameters, 11,178,564 of them will be trained
Epoch 1/10: train loss 1.2815, train accuracy 0.4150, test loss 1.2403, test accuracy 0.4683
Epoch 2/10: train loss 0.5128, train accuracy 0.8874, test loss 1.1808, test accuracy 0.5079
Epoch 3/10: train loss 0.2805, train accuracy 0.9111, test loss 1.1517, test accuracy 0.5714
Epoch 4/10: train loss 0.2171, train accuracy 0.9209, test loss 1.2179, test accuracy 0.5397
Epoch 5/10: train loss 0.1988, train accuracy 0.9249, test loss 1.3139, test accuracy 0.5159
Epoch 6/10: train loss 0.1568, train accuracy 0.9209, test loss 1.3125, test accuracy 0.5238
Epoch 7/10: train loss 0.1504, train accuracy 0.9249, test loss 1.3296, test accuracy 0.5079
Epoch 8/10: train loss 0.1372, train accuracy 0.9249, test loss 1.3465, test accuracy 0.5079
Epoch 9/10: train loss 0.1407, train accuracy 0.9209, test loss 1.3756, test accuracy 0.5238
Epoch 10/10: train loss 0.1366, train accuracy 0.9308, test loss 1.3818, test accuracy 0.4683

trained in 6.6 minutes
train accuracy 94.1%
test  accuracy 46.8%
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\run_curves.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\run_confusion.png
worst_examples is not written yet (Problem 5), so there is no run_worst.png this time
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\run_model.pt

One line for your experiment table:
| run | ImageNet | all layers | 10 | 0.0001 | 11,178,564 | 94.1% | 46.8% |

To put this model on your web page: python src/export_web.py --tag run
```

```
reading C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean
632 images, 4 classes, read in 27 s
  1980s_1990s_korean_album_covers 146
  2000s_korean_album_covers 144
  2010s_korean_album_covers 167
  2020s_korean_album_covers 175
training on 506, testing on 126
model has 11,178,564 parameters, 11,178,564 of them will be trained
Epoch 1/10: train loss 1.3574, train accuracy 0.3478, test loss 1.3681, test accuracy 0.3333
Epoch 2/10: train loss 1.0518, train accuracy 0.6225, test loss 1.4272, test accuracy 0.3413
Epoch 3/10: train loss 0.8091, train accuracy 0.7451, test loss 1.3261, test accuracy 0.4524
Epoch 4/10: train loss 0.5618, train accuracy 0.8182, test loss 1.4083, test accuracy 0.4286
Epoch 5/10: train loss 0.4148, train accuracy 0.8854, test loss 1.9774, test accuracy 0.3968
Epoch 6/10: train loss 0.3677, train accuracy 0.8794, test loss 2.2630, test accuracy 0.4286
Epoch 7/10: train loss 0.3232, train accuracy 0.8933, test loss 1.6745, test accuracy 0.4365
Epoch 8/10: train loss 0.3150, train accuracy 0.8794, test loss 1.8892, test accuracy 0.4206
Epoch 9/10: train loss 0.2450, train accuracy 0.9190, test loss 1.8572, test accuracy 0.5000
Epoch 10/10: train loss 0.2164, train accuracy 0.9170, test loss 1.7261, test accuracy 0.5079

trained in 7.6 minutes
train accuracy 93.3%
test  accuracy 50.8%
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\scratch_curves.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\scratch_confusion.png
worst_examples is not written yet (Problem 5), so there is no scratch_worst.png this time
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\scratch_model.pt

One line for your experiment table:
| scratch | random | all layers | 10 | 0.0001 | 11,178,564 | 93.3% | 50.8% |
```

```
To put this model on your web page: python src/export_web.py --tag scratch
```

```
reading C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean
632 images, 4 classes, read in 26 s
  1980s_1990s_korean_album_covers 146
  2000s_korean_album_covers 144
  2010s_korean_album_covers 167
  2020s_korean_album_covers 175
training on 506, testing on 126
model has 11,178,564 parameters, 2,052 of them will be trained
Epoch 1/10: train loss 1.4338, train accuracy 0.2846, test loss 1.3914, test accuracy 0.3571
Epoch 2/10: train loss 1.3012, train accuracy 0.3893, test loss 1.3600, test accuracy 0.3889
Epoch 3/10: train loss 1.1884, train accuracy 0.4763, test loss 1.3422, test accuracy 0.3571
Epoch 4/10: train loss 1.1217, train accuracy 0.5099, test loss 1.3782, test accuracy 0.3968
Epoch 5/10: train loss 1.0985, train accuracy 0.5455, test loss 1.3074, test accuracy 0.4127
Epoch 6/10: train loss 1.0378, train accuracy 0.6067, test loss 1.3062, test accuracy 0.4365
Epoch 7/10: train loss 0.9855, train accuracy 0.6403, test loss 1.2967, test accuracy 0.4206
Epoch 8/10: train loss 0.9610, train accuracy 0.6285, test loss 1.3056, test accuracy 0.4127
Epoch 9/10: train loss 0.9300, train accuracy 0.6403, test loss 1.2901, test accuracy 0.4603
Epoch 10/10: train loss 0.8968, train accuracy 0.6739, test loss 1.2939, test accuracy 0.4365

trained in 4.9 minutes
train accuracy 71.9%
test  accuracy 43.7%
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\frozen_curves.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\frozen_confusion.png
worst_examples is not written yet (Problem 5), so there is no frozen_worst.png this time
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\frozen_model.pt

One line for your experiment table:
| frozen | ImageNet | last layer | 10 | 0.001 | 2,052 | 71.9% | 43.7% |

To put this model on your web page: python src/export_web.py --tag frozen
```

```
reading C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean
632 images, 4 classes, read in 32 s
  1980s_1990s_korean_album_covers 146
  2000s_korean_album_covers 144
  2010s_korean_album_covers 167
  2020s_korean_album_covers 175
training on 506, testing on 126
model has 11,178,564 parameters, 11,178,564 of them will be trained
Epoch 1/10: train loss 1.3223, train accuracy 0.4150, test loss 1.7054, test accuracy 0.4444
Epoch 2/10: train loss 0.4688, train accuracy 0.8439, test loss 1.4875, test accuracy 0.5000
Epoch 3/10: train loss 0.2504, train accuracy 0.9111, test loss 1.5056, test accuracy 0.5952
Epoch 4/10: train loss 0.1887, train accuracy 0.9091, test loss 1.5301, test accuracy 0.5873
Epoch 5/10: train loss 0.1714, train accuracy 0.9308, test loss 1.4661, test accuracy 0.5397
Epoch 6/10: train loss 0.1575, train accuracy 0.9209, test loss 1.4822, test accuracy 0.5476
Epoch 7/10: train loss 0.1398, train accuracy 0.9269, test loss 1.3769, test accuracy 0.5476
Epoch 8/10: train loss 0.1245, train accuracy 0.9269, test loss 1.4695, test accuracy 0.5397
Epoch 9/10: train loss 0.1290, train accuracy 0.9150, test loss 1.5302, test accuracy 0.5397
Epoch 10/10: train loss 0.1233, train accuracy 0.9348, test loss 1.4904, test accuracy 0.5397

trained in 12.4 minutes
train accuracy 94.1%
test  accuracy 54.0%
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\lr3e4_curves.png
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\lr3e4_confusion.png
worst_examples is not written yet (Problem 5), so there is no lr3e4_worst.png this time
wrote C:\Users\신혜원\Documents\mas1004-assignment1-new\results\lr3e4_model.pt

One line for your experiment table:
| lr3e4 | ImageNet | all layers | 10 | 0.0003 | 11,178,564 | 94.1% | 54.0% |

To put this model on your web page: python src/export_web.py --tag lr3e4
```

### Output of `check.py`

This is the complete saved four-class check, without rerunning evaluation or deleting its historical warning.

```
MAS1004 Assignment 1, checking your work

Training images  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\clean)
------------------------------------------------------------------------
  1980s_1990s_korean_album_covers    93 images
  2000s_korean_album_covers       79 images
  2010s_korean_album_covers       87 images
  2020s_korean_album_covers      114 images

Your images from a new source  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\new_images)
------------------------------------------------------------------------------------------------
  1980s_1990s_korean_album_covers     8 images
  2000s_korean_album_covers        6 images
  2010s_korean_album_covers        8 images
  2020s_korean_album_covers        5 images

The same image appearing twice
------------------------------
  ok       no file appears twice byte for byte. Near copies (the same picture resized or re-saved)
  ↪  are only found by `python src/clean.py suspects` and `python src/clean.py overlap`.

The model you exported for the web page
---------------------------------------
  ok       4 classes: 1980s_1990s_korean_album_covers, 2000s_korean_album_covers,
  ↪  2010s_korean_album_covers, 2020s_korean_album_covers
  ok       model.onnx is 44.8 MB, 11,178,564 parameters
           input [1, 3, 224, 224], prepared as: shorter side 256, centre 224 x 224, mean [0.485,
           ↪  0.456, 0.406], std [0.229, 0.224, 0.225]
  warning  docs/model.onnx is not committed. Your published page will not have a model until you
  ↪  commit and push it.

Does the exported model give the answers Python gave?
-----------------------------------------------------
  ok       yes, they agree (largest gap 0.0000)

Your images from a new source  (C:\Users\신혜원\Documents\mas1004-assignment1-new\data\new_images)
------------------------------------------------------------------------------------------------

  accuracy on your new images: 14/27 = 51.9%

  rows are what it really is, columns are what the model said
                                  1980s_19 2000s_ko 2010s_ko 2020s_ko
    1980s_1990s_korean_album_covers        5        1        1        1
    2000s_korean_album_covers              1        4        1        0
    2010s_korean_album_covers              0        3        2        3
    2020s_korean_album_covers              2        0        0        3

  the mistakes it was most sure about:
    2013.webp                    said 2000s_korean_album_covers (96%), really
    ↪  2010s_korean_album_covers
    2012'.webp                   said 2020s_korean_album_covers (89%), really
    ↪  2010s_korean_album_covers
    1985.jpg                     said 2000s_korean_album_covers (87%), really
    ↪  1980s_1990s_korean_album_covers
    2012.webp                    said 2020s_korean_album_covers (82%), really
    ↪  2010s_korean_album_covers
    2014.webp                    said 2000s_korean_album_covers (72%), really
    ↪  2010s_korean_album_covers

Summary
-------
  nothing is broken
  1 thing(s) worth a sentence in your report:
```

```
- docs/model.onnx is not committed. Your published page will not have a model until you commit
↪  and push it.
```

## Supporting evidence

Saved outputs below retain their original text, language and warnings. Only report copies of UTF-16 logs were converted to UTF-8. The exact Korean P7 requests and untranslated evidence are the exceptions to the report's English prose.

### Saved model and dataset consistency inspection

```
Current readable clean counts: {"1980s_1990s_korean_album_covers": 93, "2000s_korean_album_covers":
↪  79, "2010s_korean_album_covers": 87, "2020s_korean_album_covers": 114}
Current clean total: 373; saved total: 373
Current test split matches saved paths: True; test count: 75
Clean images modified after model: 0
CLEAN retraining needed: False
Historical content hashes were not saved with training; count/split/timestamps establish
↪  consistency, not proof of identical bytes.
Existing CLEAN model retained; no retraining.
Running: uv run python src/check.py --compare run clean | tee results/compare.txt
Comparison command exit code: 1 (see full report for any non-comparison checks)
STEP13 FINISHED
```

### Accuracy differences calculated from final printed values

```
run: 훈련 94.1% - 테스트 46.8% = 47.3 퍼센트포인트
scratch: 훈련 93.3% - 테스트 50.8% = 42.5 퍼센트포인트
frozen: 훈련 71.9% - 테스트 43.7% = 28.2 퍼센트포인트
lr3e4: 훈련 94.1% - 테스트 54.0% = 40.1 퍼센트포인트
clean: 훈련 100.0% - 테스트 56.0% = 44.0 퍼센트포인트
scratch - run: 50.8% - 46.8% = 4.0 퍼센트포인트
run - frozen: 46.8% - 43.7% = 3.1 퍼센트포인트
lr3e4 - run: 54.0% - 46.8% = 7.2 퍼센트포인트
```

### Asset and preprocessing audit

```
삭제 로그 행 수: 196
의심 이미지 행 수: 632
근접 복사 쌍 행 수: 183
1980s_1990s_korean_album_covers: 상위 20개 중 로그에 기록된 제외 7
2000s_korean_album_covers: 상위 20개 중 로그에 기록된 제외 12
2010s_korean_album_covers: 상위 20개 중 로그에 기록된 제외 8
2020s_korean_album_covers: 상위 20개 중 로그에 기록된 제외 9
로그 날짜별 행 수: {'2026-10-12': 167, '2026-10-13': 29}
첫 단계 기록: 85
P2 원본: data\clean\1980s_1990s_korean_album_covers\1985.jpg
원본 크기: (170, 170)
확대 크기: (256, 256)
RGB 자르기 크기: (224, 224)
정규화 역변환 RGB 픽셀 완전 일치: True
기존 P2 비교 PNG 읽기 검증: 통과 (기존 그림 재생성 없음)
웹 클래스 수: 4
웹 자체검사 사례 수: 3
ONNX 파일 바이트: 44786703
소스 및 짧은 보고서 SHA256 변경 없음: 통과
```